PROJECT DOCUMENTATION

TickTracker UK

A full-stack application for exploring tick sightings and collecting community reports across the UK.

Project overview

TickTracker UK brings together sightings from Elanco's educational API and reports submitted through the web interface. Users can search records, explore supported cities on a map, review activity summaries, and learn about tick species and prevention. The project is an MVP built as a single Spring Boot service with a locally persisted H2 database.

At a glance

Area	Implementation
Backend	Java 17, Spring Boot 4, Spring MVC, Spring Data JPA, Jakarta Validation
Storage	File-based H2 database at data/ticktracker
Frontend	HTML, CSS, and browser JavaScript served by Spring Boot
Map	Leaflet with OpenStreetMap tiles and approximate city-centre markers
External data	Elanco tick-sightings API at dev-task.elancoapps.com

Core user experience

- Explore reported sightings on a UK map and in a searchable list.
- Filter records by date range, city, and species; review city counts and weekly or monthly trends.
- Open a city detail panel with a recent timeline, directions, and sharing actions.
- Read species identification and tick-bite prevention information, then submit a community report.

The application uses city-level locations. Map markers represent approximate city centres and should not be interpreted as exact sighting locations.

Architecture and data flow

The browser loads the static interface from Spring Boot and calls REST endpoints on the same origin. Controllers delegate application logic to services. The JPA repository persists sightings and runs filtered and aggregate queries against H2.

Request and data paths

Flow	What happens
Initial import	On application startup, if the sighting table is empty, the initializer requests the Elanco dataset. The application remains available if the external request fails.
Import processing	Records without an ID, date, location, or species are skipped. Repeated IDs in the same response and IDs already in the database are skipped. Valid records are stored with source ELANCO_API.
Community report	The report endpoint validates required fields and length limits, assigns a generated ID, marks the source USER_REPORTED, and stores the record.
Search	Optional date bounds and case-insensitive city and species filters are applied by the repository. Results are ordered newest first.
Analytics	Sightings are aggregated by city or grouped into ISO weeks or calendar months; trends can optionally be limited to a city.

Main components

Component	Responsibility
SightingController	Search, create, and manually import sightings.
SightingService	Validate and normalize report/import data; prevent duplicate imports.
ElancoApiClient	Call the external API and convert upstream/network failures into application errors.
AnalyticsController / AnalyticsService	Expose city totals and weekly/monthly trends.
GlobalExceptionHandler	Return structured responses for validation, parsing, external API, data conflict, and unexpected errors.
SightingDataInitializer	Attempt an Elanco import at startup only when the database is empty.

Persistence

H2 stores data in the project's data folder. Deleting its database files removes both imported sightings and community reports. The next startup attempts a fresh external import if the database is empty.

Frontend and API reference

The dashboard is served from src/main/resources/static. Its JavaScript requests sightings and analytics from the backend and rebuilds the list, summary cards, and map markers when filters change.

Frontend sections

Section	Behavior
Dashboard	Summary counts, filters, map, city summaries, and trend summaries.
Map details	A selected city shows recent records, a timeline, directions, and share actions. The map uses a fixed set of supported city coordinates.
Education	Species guide, prevention tips, identification caution, and links to further resources.
Report form	Required date, location, and species fields; optional scientific name; validation and success/error feedback.

HTTP endpoints

Method and path	Purpose
GET /api/health	Backend health status.
GET /api/sightings	List sightings; accepts optional fromDate, toDate, location, and species.
POST /api/sightings	Create a community sighting report.
POST /api/sightings/import	Manually retry or run the idempotent Elanco import.
GET /api/elanco/sightings	Fetch current Elanco data without storing it.
GET /api/analytics/cities	Return sighting counts grouped by city.
GET /api/analytics/trends?period=weekly	Return weekly trends; use period=monthly for calendar months and optionally pass location.

Example requests

Search by date and city

```
/api/sightings?fromDate=2026-01-01T00:00:00&toDate=2026-01-31T23:59:59&location=London
```

Create a report (date, location, and species are required)

```
{"date":"2026-10-31T12:30:00","location":"London","species":"Sheep or deer
tick","latinName":"Ixodes ricinus"}
```

Running and extending the project

Requirements

- JDK 17.
- Internet access for the Elanco API, Leaflet assets, and OpenStreetMap map tiles.
- Maven is included through the project wrapper.

Start locally

Run from the repository root on Windows:

```
.\mvnw.cmd spring-boot:run
```

On macOS or Linux:

```
./mvnw spring-boot:run
```

Then open http://localhost:8080. Startup import totals are logged. If Elanco is unavailable, retry later with POST /api/sightings/import.

Current limits

- The map recognizes only a predefined set of city names. Other locations remain searchable but are not mapped.
- All search and analytics data is loaded without pagination; larger datasets may require paging and database-side aggregation.
- City is the current geographic grouping. Coordinates are approximate and do not identify precise tick locations.
- Reports are stored locally; there are no user accounts or image uploads.
- An unavailable Elanco service prevents initial import, though the application remains available and supports a manual retry.

Potential improvements

Add pagination and batch import, configurable geocoding, automated coverage for import and validation, image storage if needed, deployment configuration for a managed database, and a repeatable deployment pipeline. These are future extensions rather than requirements for the current MVP.

Project source

The source repository contains the Spring Boot application, static frontend, Maven Wrapper, and local configuration. The public project repository can be linked in the final application submission.